

EC06 — CI/CD avec GitHub Actions et Docker

Bloc 3 · Architectures · Évaluation individuelle · Durée : 4h00 · Épreuve sur ordinateur

Avant de commencer — lisez attentivement

Cette évaluation porte sur les compétences : Git, Docker, CI/CD.

- Chaque critère est noté indépendamment — un pilier non terminé n'empêche pas de valider les autres.
- La note de validation correspond à la moitié des points (30/60).
- Les durées indiquées dans énoncé ne sont là qu'à titre indicatif
- Mieux vaut un pipeline propre et partiel qu'un pipeline complet mais cassé.

Si on bloque sur une étape, on explique son raisonnement dans le `README.md` : les choix justifiés rapportent des points.

Contexte



TrustiPro est une marketplace en cours de lancement qui met en relation des particuliers avec des artisans qualifiés (plombiers, électriciens, peintres, menuisiers, carreleurs). La plateforme repose sur une API Node.js, un front React et une base de données PostgreSQL. Elle héberge des fichiers sensibles (justificatifs professionnels des artisans) et envoie des notifications email à chaque étape clé du parcours utilisateur.

La plateforme n'est pas encore en production : elle tourne sur un VPS d'entrée de gamme utilisé comme environnement de recette. Le développeur DevOps récemment recruté a pour première mission de mettre en place une chaîne CI/CD qui garantisse la qualité du code avant tout déploiement et automatise la mise en production.

La mission : mettre en place cette chaîne CI/CD complète autour d'une petite API Node.js fournie, en utilisant Git, GitHub, GitHub Actions et Docker.

Application fournie

On dispose d'une archive `EC06_TrustiPro.zip` contenant une mini API Express (Node.js 20) entièrement fonctionnelle :

```
trustipro-api/
├─ src/
│  ├─ server.js      # point d'entrée, démarre le serveur HTTP
│  ├─ app.js         # construction de l'app Express (routes, JSON, 404)
│  ├─ health.js      # fonction getHealth() : statut API + test connexion PostgreSQL
│  ├─ db.js          # pool de connexion PostgreSQL (pg) et test de connexion
│  └─ artisans.js    # fonction getArtisans() : liste fictive en mémoire
├─ tests/
│  ├─ app.test.js    # tests sur GET / et la gestion des routes 404
│  ├─ health.test.js # tests sur GET /health
│  └─ artisans.test.js # tests sur GET /artisans
├─ eslint.config.js  # config ESLint flat (recommended)
├─ package.json      # scripts : start, dev, test, lint
└─ README.md         # minimal, sans instruction de déploiement
```

L'API expose trois endpoints :

Méthode	Route	Description
GET	<code>/</code>	Informations sur l'API et liste des endpoints disponibles
GET	<code>/health</code>	Statut de l'API et de la connexion à PostgreSQL (<code>"connected"</code> ou <code>"unavailable"</code> , sans jamais faire échouer la requête)
GET	<code>/artisans</code>	Liste fictive d'artisans (données statiques en mémoire)

Le code applicatif n'est pas évalué. Des modifications sont possibles si nécessaire, mais l'épreuve évalue uniquement la chaîne CI/CD construite autour.

Les fichiers suivants sont intentionnellement absents — c'est à vous de les créer :

- `Dockerfile` et `.dockerignore`
- `docker-compose.yml`
- `.env` et `.env.dist`
- `.gitignore`
- `.github/workflows/ci.yml`
- toute initialisation Git

Travail à réaliser

Pilier 1 — Versioning Git (~20 min)

1.1 Initialisation du dépôt

On initialise un dépôt Git local dans le dossier de l'application.

On crée un dépôt public sur son compte GitHub et on y pousse le projet.

On ajoute un fichier `.gitignore` adapté à la stack Node.js (au minimum : `node_modules/`, `.env`).

On réalise un premier commit avec un message au format Conventional Commits (ex. `chore: init project structure`).

1.2 Stratégie de branches

On met en place une stratégie GitFlow simplifiée :

Branche	Rôle
<code>main</code>	code stable, protégé en production
<code>dev</code>	intégration des features
<code>feature/<nom></code>	travaux en cours, éphémères

On documente cette stratégie dans le `README.md` (2 à 3 phrases suffisent).

1.3 Pull Request

On ouvre et fusionne au moins une PR pendant l'épreuve (par exemple `feature/dockerfile` ou `feature/ci-pipeline`).

La PR doit comporter :

- un titre clair,
- une description structurée : contexte, changements apportés, comment tester.

La CI se déclenchera automatiquement sur les commits de la branche `feature` — son statut sera visible dans la PR.

Pilier 2 — Conteneurisation Docker (~1h00)

2.1 Dockerfile multistage

On écrit un `Dockerfile` respectant les contraintes suivantes :

- Étape `builder` : installe les dépendances (`npm ci`) et prépare l'application.
- Étape finale : image légère (`node:20-alpine` ou équivalent), ne contenant que le strict nécessaire à l'exécution.
- Utilisateur non-root explicitement défini (`USER node` ou équivalent).
- Instruction `EXPOSE` pour le port utilisé (3000).

- Instruction `HEALTHCHECK` qui appelle `GET /health`.

Bonus : image finale inférieure à 200 Mo (vérifiable avec `docker images`).

2.2 docker-compose.yml

On fournit un `docker-compose.yml` permettant de démarrer l'application avec `docker compose up`.

Il doit inclure :

- le service applicatif (`app`) basé sur le `Dockerfile`,
- un service de base de données (`db`) — PostgreSQL,
- un volume nommé pour la persistance des données,
- le chargement des variables d'environnement via `env_file: .env`.

2.3 Gestion des fichiers d'environnement

- `.env.dist` : versionné dans Git, sans valeur secrète réelle, sert de modèle.
- `.env` : non versionné (présent dans `.gitignore`), contient les valeurs locales.

Exemple de `.env.dist` attendu :

```
PORT=3000
NODE_ENV=production
POSTGRES_USER=
POSTGRES_PASSWORD=
POSTGRES_DB=
DB_URL=
```

Pilier 3 — Pipeline GitHub Actions (~1h40)

On crée un workflow `.github/workflows/ci.yml` déclenché sur chaque push sur toutes les branches.

Le workflow doit comporter trois jobs distincts :

3.1 Job `quality` — lint sur le runner + tests dans Docker

Le lint est exécuté directement sur le runner. Les tests sont exécutés via Docker.

Pattern attendu dans le job :

```
- name: Installer les dépendances
  run: npm ci

- name: Lint
  run: npm run lint

- name: Copier le fichier d'environnement
  run: cp .env.dist .env

- name: Tests
  run: docker compose run --rm app npm test
```

Le job doit échouer si le lint ou les tests ne passent pas.

On publie les résultats de tests comme artefact du workflow (fichier de rapport Jest ou log).

3.2 Job `build` — construction de l'image Docker

On construit l'image avec `docker build` ou `docker compose build`.

On tague l'image avec le SHA court du commit (`${{ github.sha }}` tronqué).

On utilise les secrets GitHub pour tout credential éventuel.

Bonus : push de l'image vers GitHub Container Registry (`ghcr.io`) uniquement sur `main`.

3.3 Job `deploy` — déploiement simulé (sur `main` uniquement)

Le déploiement est simulé via un script `deploy.sh` qui affiche les commandes qu'il aurait exécutées et écrit un fichier `deploy.log`.

On publie `deploy.log` comme artefact du workflow.

3.4 Bonnes pratiques obligatoires

Aucun secret en clair dans le code, dans `ci.yml` ou dans les logs.

Les secrets sont stockés dans GitHub Secrets et injectés via `${{ secrets.NOM_SECRET }}`.

Bonus : cache des dépendances npm, badge de statut CI dans le `README.md`, trigger `pull_request` en plus du `push`.

Pilier 4 — Gestion des secrets (~10 min)

Ce pilier conditionne la sécurité de tout le pipeline. Il est court à mettre en place mais éliminatoire si négligé.

4.1 Créer les secrets dans GitHub

Dans le dépôt GitHub, on crée les secrets suivants :

Nom du secret	Valeur attendue	Utilisé dans
<code>POSTGRES_USER</code>	nom d'utilisateur BDD (ex. <code>trustipro</code>)	<code>docker-compose.yml</code> via CI
<code>POSTGRES_PASSWORD</code>	mot de passe BDD (ex. <code>changeme</code>)	<code>docker-compose.yml</code> via CI
<code>POSTGRES_DB</code>	nom de la base (ex. <code>trustipro_db</code>)	<code>docker-compose.yml</code> via CI

4.2 Injecter les secrets dans le workflow CI

Les secrets ne sont jamais écrits en clair. Ils sont injectés dans le job via la section `env` :

```

jobs:
  quality:
    runs-on: ubuntu-latest
    env:
      POSTGRES_USER: ${ secrets.POSTGRES_USER }
      POSTGRES_PASSWORD: ${ secrets.POSTGRES_PASSWORD }
      POSTGRES_DB: ${ secrets.POSTGRES_DB }
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint
      - run: cp .env.dist .env
      - run: docker compose run --rm app npm test

```

GitHub masque automatiquement les valeurs des secrets dans les logs — on verra `***` à la place. On ne fait jamais un `echo ${ secrets.MA_VALEUR }` : cela tenterait d'afficher le secret et GitHub le bloquerait.

4.3 Lien entre `.env.dist` et les secrets CI

Le `.env.dist` versionné contient les clés mais pas les valeurs :

```

# .env.dist – versionné, sans valeur secrète
PORT=3000
NODE_ENV=production
POSTGRES_USER=
POSTGRES_PASSWORD=
POSTGRES_DB=
DB_URL=

```

En CI, l'étape `cp .env.dist .env` crée un `.env` vide. Les vraies valeurs arrivent via les variables d'environnement du job (`env:`), qui sont elles-mêmes alimentées par les GitHub Secrets. Le `docker-compose.yml` lit le `.env` et les variables d'environnement du runner — les deux se complètent.

4.4 Vérifications obligatoires

Avant de pousser le premier commit, on vérifie :

```

# .env doit apparaître dans .gitignore
cat .gitignore | grep .env

# .env ne doit PAS être tracké par Git
git status          # .env ne doit pas apparaître
git ls-files .env   # doit retourner vide

```

Si `.env` a déjà été commité par erreur :

```

git rm --cached .env
git commit -m "chore: remove .env from tracking"

```

`.env` versionné avec des valeurs réelles = 0 point sur les 4 points secrets, indépendamment du reste du pipeline.

Rapport — README.md (~50 min)

Le `README.md` est avant tout un document de restitution technique. Il sert également de page d'accueil du dépôt GitHub. C'est le document qui permet d'évaluer la compréhension de l'ensemble de la chaîne mise en place — pas seulement si le pipeline passe au vert.

Un README qui se limite à des commandes sans expliquer les choix effectués ne rapporte pas de points sur ce critère, même si le pipeline fonctionne.

On rédige ce document au fil de l'eau, pas en une seule fois à la fin, et on réserve un temps de relecture finale pour vérifier que chaque attendu de l'épreuve y figure bien.

Squelette minimal attendu

La structure ci-dessous est fournie à titre de cadre. On peut la modifier librement du moment que tous les attendus y figurent.

TrustiPro – API CI/CD

URL du dépôt : `https://github.com/<username>/<repo>`

1. Workflow Git

Décrire la stratégie de branches retenue et la justifier en 2-3 phrases.
Expliquer comment la PR a été utilisée pendant l'épreuve.

2. Conteneurisation Docker

Décrire le Dockerfile multistage : image de base choisie, pourquoi un utilisateur non-root, ce que fait le HEALTHCHECK.

3. Pipeline CI/CD

Insérer ici le schéma Mermaid représentant les trois jobs et leurs déclencheurs.

Décrire en quelques phrases le rôle de chaque job et ce qui déclenche le job deploy uniquement sur main.

4. Gestion des secrets

Lister les secrets utilisés (noms uniquement, pas les valeurs).
Expliquer où ils sont définis et comment ils sont injectés dans la CI.
Confirmer explicitement que `.env` n'est pas versionné et comment on l'a vérifié.

5. Lancement et utilisation du projet

Donner les commandes nécessaires pour cloner le dépôt et lancer le projet en local, de façon à ce qu'un tiers puisse reproduire l'exécution sans aide supplémentaire :

```
git clone https://github.com/<username>/<repo>.git
cd <repo>
cp .env.dist .env
docker compose up
```

Indiquer également comment vérifier que l'application fonctionne (ex. quelle URL appeler, quelle réponse attendre sur /health).

6. Ce qui n'a pas été fait / améliorations envisagées

Lister honnêtement ce qui manque et ce qui pourrait être amélioré.

Le schéma Mermaid de la section 3 est obligatoire. Whimsical ou Excalidraw sont également acceptés (export image intégré dans le README). Un schéma absent ou illisible ne rapporte pas les points associés.

Livrables

On remet :

1. L'URL du dépôt GitHub, indiquée dans le `README.md`.
2. Une archive `EC06_NomPrenom.zip` contenant :

```
EC06_NomPrenom.zip
├─ (code complet avec .git/)
├─ Dockerfile
├─ .dockerignore
├─ docker-compose.yml
├─ .env.dist
├─ .gitignore
├─ .github/
│   └─ workflows/
│       └─ ci.yml
├─ deploy.sh
└─ README.md
```

Le dépôt GitHub public fait foi pour les preuves de runs CI et de PR fusionnée — aucune capture d'écran n'est demandée.

Barème (/60)

Critère	Points
Pilier 1 — Init Git, <code>.gitignore</code> (avec <code>.env</code>), stratégie de branches documentée, 1 PR fusionnée avec description	8
Pilier 2a — <code>Dockerfile</code> multistage, image légère, utilisateur non-root, <code>HEALTHCHECK</code> , <code>EXPOSE</code>	10
Pilier 2b — <code>docker-compose.yml</code> fonctionnel (<code>docker compose up</code> démarre l'app)	6

Critère	Points
Pilier 3a — Job <code>quality</code> : lint sur le runner, tests dans Docker, artefact publié	6
Pilier 3b — Job <code>build</code> : image construite, tags cohérents	8
Pilier 3c — Job <code>deploy</code> : script <code>deploy.sh</code> fonctionnel, déclenché sur <code>main</code> , <code>deploy.log</code> publié	8
Pilier 4 — Gestion des secrets (<code>.env</code> non versionné, secrets dans GitHub Secrets, pas de fuite)	4
Qualité du code (lisibilité et rigueur du Dockerfile, docker-compose.yml, deploy.sh)	2
Rapport <code>README.md</code> (restitution technique, schéma Mermaid, justifications des choix)	8
Total	60

Bonus	Points
Qualité des commits (messages conventionnels, atomicité, lisibilité de l'historique)	+2
Push vers GitHub Container Registry, cache npm, badge CI, image < 200 Mo	jusqu'à +6
Total avec bonus	68

Validation : $\geq 30/60$ · Rattrapage : 21–29 · Non validé : < 21

Contraintes et interdictions

Imposé	GitHub + GitHub Actions + Docker
Imposé	<code>Dockerfile</code> multistage, image finale légère, utilisateur non-root
Imposé	Au moins 1 PR fusionnée via GitHub
Interdit	<code>.env</code> versionné avec des valeurs réelles
Interdit	Secrets en clair dans <code>ci.yml</code> ou dans le code
Interdit	Image Docker contenant des credentials
Autorisé	Documentations officielles : Docker, GitHub Actions, Node.js, etc.
Autorisé	Outils de schéma (Whimsical, Excalidraw, Mermaid)
Autorisé	Extensions IDE (linter, Docker, Git)
Interdit	ChatGPT, Copilot, Claude, tout outil de génération de code

Conseils pour gérer le temps

1. Lire l'énoncé en entier avant de commencer (5 min) : la structure en quatre piliers permet de répartir l'effort sans tout faire dans l'ordre imposé.
2. Commencer par Docker : si `docker compose up` ne démarre pas l'app en local, il ne fonctionnera pas non plus en CI.
3. Ne modifier le code fourni qu'en cas d'adaptation obligatoire. Le code n'est pas noté.
4. Vérifier `.env` et `.env.dist` avant de pousser : `.env` doit être dans `.gitignore`.
5. Travailler par petits commits : un commit qui passe la CI vaut mieux qu'un gros commit qui échoue.
6. Rédiger le `README.md` au fil de l'eau, pas en une seule fois à la fin, et réserver un temps de relecture dédié en fin d'épreuve pour vérifier que chaque section couvre bien les attendus correspondants.
7. En cas de manque de temps : sacrifier les bonus et s'assurer que les secrets sont correctement gérés et que le pipeline passe au vert.